

Compiler

Source Program in high level language.

```
int main() {
```

```
int a = 10, b = 5, c;
```

```
c = a + b;
```

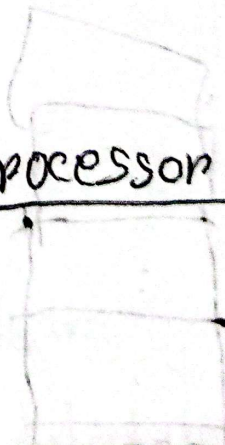
```
printf("%d", c);
```

```
return;
```

3.

* Analysis of source - program.

Microprocessor



these

SAD

Sog-2

V-model:

Overall process
(Compiler + interpreter + assembler) Compiler

include <stdio.h>

int main() {

int a = 10, b = 5, c;

c = a + b - 1 * 5.0

printf("%i, %d", c);

return 0;

lexeme token pattern:

int keyword datatype:

identification: main, a, b, c, printf.

keyword: int, return.

operator: =, +, -, *

delimiter/symbols/separator: (,), {, }, ' ', ;

constant: 10, 5, 1, 5.0, "%i, %d".

lexical Analysis

two tools are used
scanner, tokenizer.

16.09.25

Symbol Table

	id	name
1	id ₁	a
2	id ₂	b
3	id ₃	c
4		

id₁ = 10

id₂ = 5

id₃ = id₁ + id₂ - 1 + 0.05c

Syntax Analysis

→ CFG

→ parse tree

→ Syntax tree

* Diagram: phase of a compiler

Syntax tree is parse tree:

CFG:

$E \rightarrow E + E$

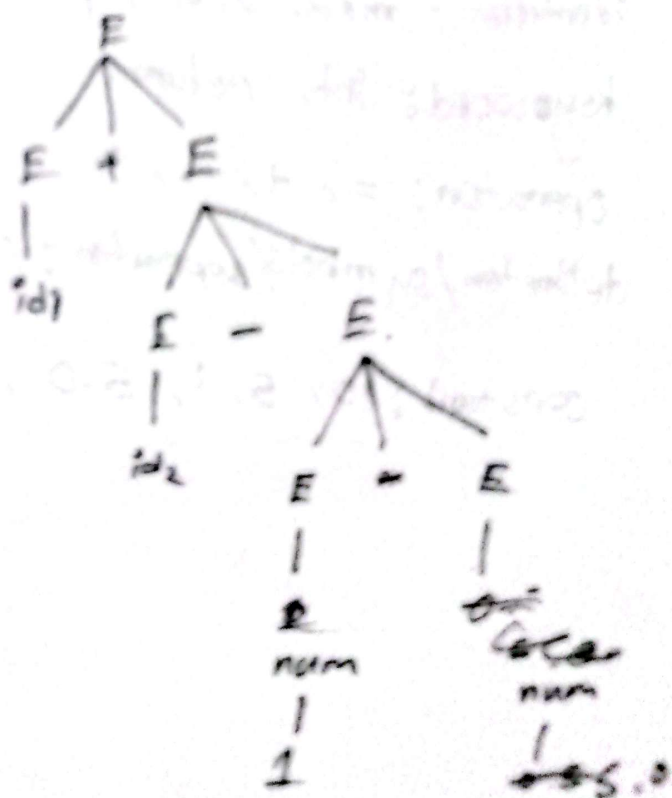
$E \rightarrow E - E$

$E \rightarrow E * E$

$E \rightarrow E / E$

$E \rightarrow (E) | id$

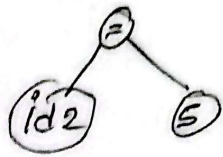
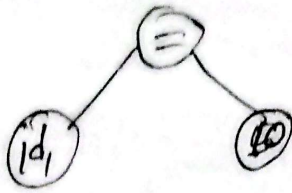
$E \rightarrow num$



$id1 = 10$
 $id2 = 5$ } Parse tree

E
 $|$
 num
 $|$
 10

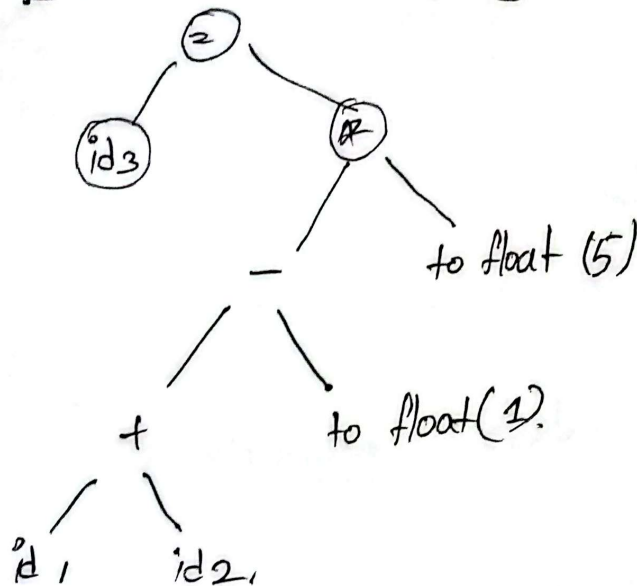
E
 $|$
 num
 $|$
 5



Syntax tree.

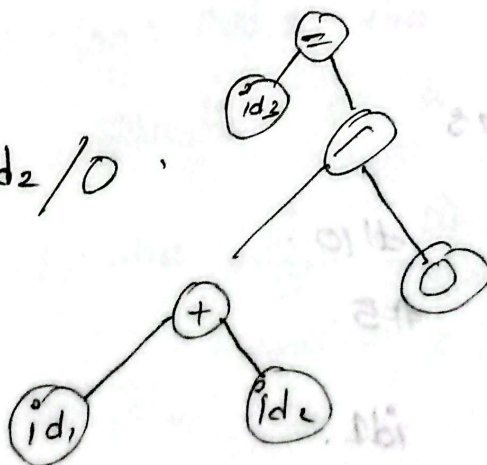
Semantic Analysis.

Ans jemon chai lemon
dite parbo.



$id3 = id1 + id2 / 0$

↓
 arithmetic error.



int $a[10]$
 $i = a[11]$
 array out of bound.

$$id_1 = 10.$$

$$id_2 = 5.$$

$$t_1 = id_1 + id_2.$$

$$t_2 = \text{to float}(1)$$

$$t_3 = t_1 - t_2.$$

$$t_4 = \text{to float}(5).$$

$$t_5 = t_3 \times t_4$$

$$id_3 = t_5.$$

→ Optimized

$$id_1 = 10$$

$$id_2 = 5.$$

$$t_1 = id_1 + id_2.$$

$$t_2 = t_1 - 1.0.$$

$$id_3 = t_2 \times 5.0$$

code generations

MOV F id_1 , #10.

MOVF id_2 , #5.

MOV F R_1 , id_1 .

ADD F R_1 , id_2 .

SUB F R_1 , #1.

MUL F R_1 , #5

MOVF id_3 , R_1 .

07.10.25

Compiler

2.2 Syntax definition (Book)

CFG = syntax definition

* four components of context free grammar.

• syntax directed definition translation

CFG
 $E \rightarrow E + E$

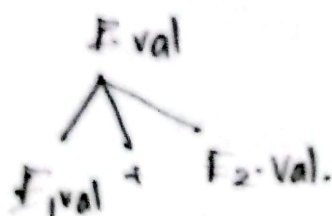
Semantic Rule

$E.val = E_1.val + E_2.val$

$E \rightarrow E_1 + E_2 \mid * +$
 $\hookrightarrow$ aggregate attribute.

• Synthesize Attribute.

• Inherited Attribute.



crs { int a, b, c. syntax definition & data type.

$D_n \rightarrow T L_n$
 $T \rightarrow \text{int} \mid \text{long} \mid \text{float}.$
 $L \rightarrow L, \text{id}.$
 $L \rightarrow \text{id}$

Syntax directed Definition

D. type T , type L in

T . type = int | ... |

L . in = L in , add type (entry for id , type).

Syntax Definition.

→ language (L)

→ RE

→ NFA

→ ϵ -NFA

→ definition: LMD, RMD

→ parse tree

→ Annotated parse

→ parse tree.

→ Ambiguity

→ Syntax directed translation

→ NFA to DFA.

* LMD, RMD, parse tree.
for given input string.

Not for mid only
final

Example:

→ C program

→ identification

→ number

→ if/else.
statement

→ while/do
while statement

CFG :

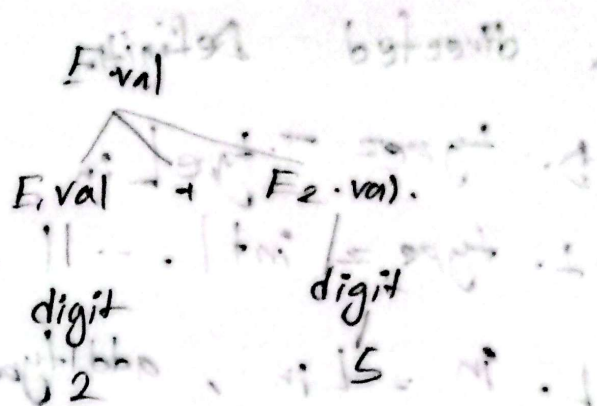
$$E \rightarrow E + E$$

$$E \rightarrow \text{digit}$$

$$\text{digit} \rightarrow 0 | \dots | 9$$

$$E.\text{val} = \text{digit.lexical}$$

$$\text{digit.lexical} = '0' | \dots | '9'$$



$$E.\text{val} = 7$$

$$E_1.\text{val} = 5$$

$$E_2.\text{val} = 2$$

digit.lexical

digit.lexical

5

2

Annotated value (comes with value)

↳ attribute + value.

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow E / E$$

$$E \rightarrow (E) \mid \text{id} \mid \text{digit}$$

$$E \rightarrow E + T$$

$$E \rightarrow E - T \mid T$$

$$T \rightarrow T * F$$

$$T \rightarrow T / F \mid F$$

$$F \rightarrow (E) \mid \text{id} \mid \text{digit}$$

How to make this
non-ambiguous.

$$5 + 2 * 1$$

$$E \rightarrow E + T$$

$$\rightarrow T + T$$

$$\rightarrow F + T$$

$$\rightarrow \text{digit} + T$$

$$\rightarrow 5 + T$$

$$\rightarrow 5 + T * F$$

$$\rightarrow 5 + 2 * F$$

$$\rightarrow 5 + \text{digit} * F$$

$$\rightarrow 5 + 2 * 1$$

$$L(\Sigma) = \{ \text{'-'} \mid a-z, A-Z, 0-9 \}$$

$$\{ \text{'-'} \mid L, D \}$$

CFG of identifier.

$$RE = \text{'-'} \mid L \mid D$$

$$RE = (\text{'-'} \mid L) (\text{'-'} \mid L \mid D)^*$$

$$= [\text{'-'} \mid a-z \mid A-Z] [\text{'-'} \mid a-z \mid A-Z \mid 0-9]^*$$

CFG:

$$I \rightarrow \text{'-'} \mid A \mid L \mid A$$

$$A \rightarrow \text{'-'} \mid A \mid L \mid A \mid D \mid A \mid \epsilon$$

$$L \rightarrow a \mid \dots \mid z \mid A \mid \dots \mid z$$

$$D \rightarrow 0 \mid \dots \mid 9$$

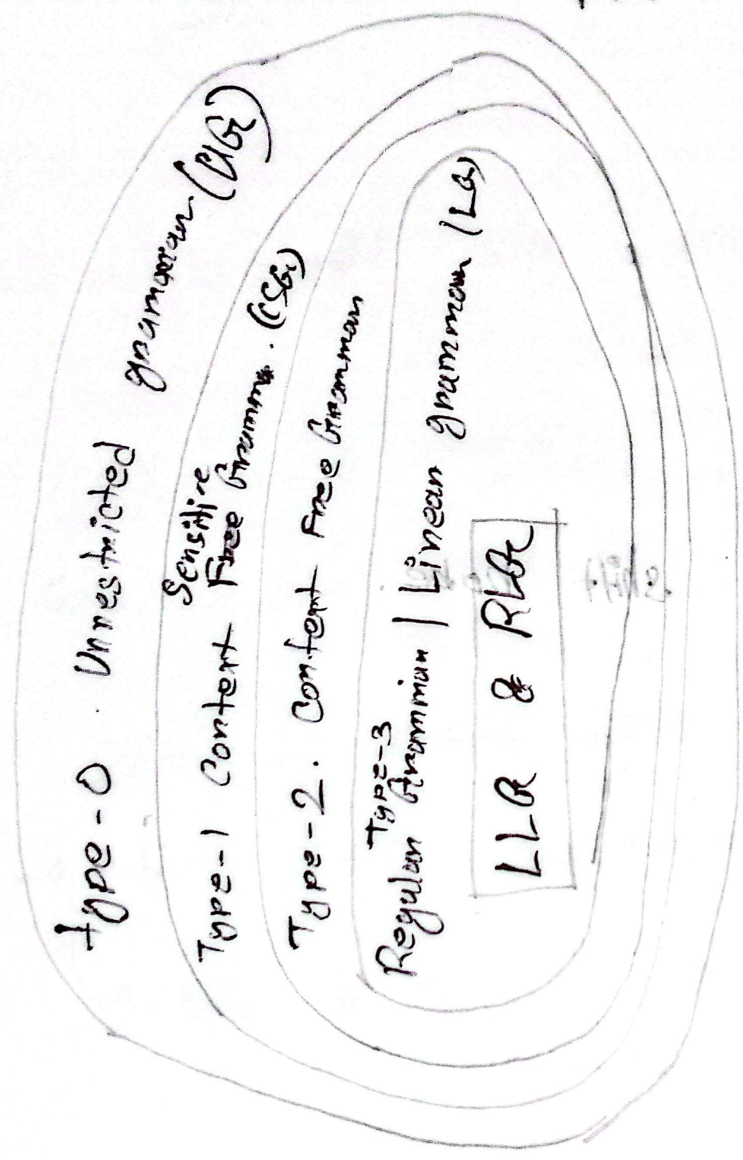
DFA vs NFA

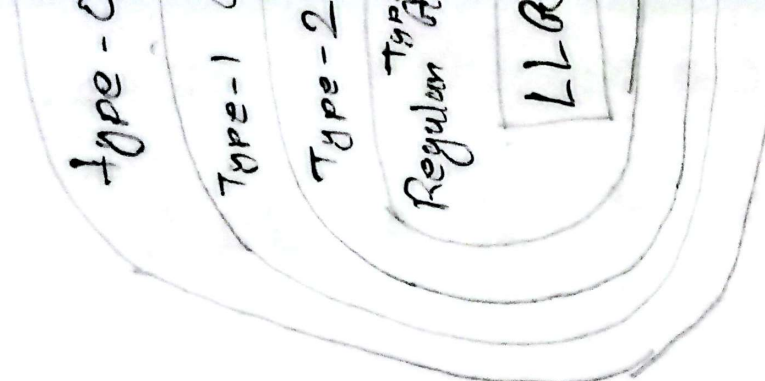
1. DFA has next step for each input
NFA doesn't need this.

2. DFA - one input one step

3. DFA has transition function.

Compiler





type - 0 (OG).

$\alpha \rightarrow \beta$ where, $\alpha \neq \epsilon$.

↓
production
here

↘ production body.

type - CSG

$\alpha, \beta \in \{V, T\}^*$

↓
jddi

$|\alpha| \leq |\beta|$

production head ϵ terminal thake.

Overall
dekhe check
dibo

$S \rightarrow A A C$
 $A A \rightarrow b B$
 $B \rightarrow d$

$A A \rightarrow abc$

↓
ed: CSG

~~Type-2: CFG.~~

Q: Why ~~CFG~~ is called so?

because production head depends on terminal and non terminal. --- (gpt diye check)

Type-2: CFG.

$\alpha \in \{V\}$, $\beta \in \{V, T\}$. $|\alpha| = 1$.

$|\alpha| \leq |\beta|$. $|\alpha| = 1$.

$S \rightarrow aAc$
 $a \rightarrow bB$
 $B \rightarrow d$

CFG.

$S \rightarrow aAc$
 $A \rightarrow bB$
 $B \rightarrow d$

CFG.

maximum

$$\left. \begin{array}{l} S \rightarrow aAc \\ a \rightarrow bB \\ B \rightarrow d \end{array} \right\} \rightarrow CSG.$$

$$\left. \begin{array}{l} S \rightarrow aAc \\ A \rightarrow bB \\ B \rightarrow d \end{array} \right\} CFG.$$

LG/RG $\Rightarrow$ production body te ~~shudh~~ ~~ekta~~ maximum
ekta. variable. thake.

All regular grammars are CFG.

$$S \rightarrow aAc$$

$$A \rightarrow bB | \epsilon.$$

$$B \rightarrow d$$

LLG: Left side e

jodi variable thake.

RLG: Right side e

jodi thake.

RG||G.

$\alpha \in \{V\}, \beta \in \{V, T\}$

$|\alpha| \leq |\beta|; |\alpha| = 1.$

$|\beta \in \{V, T\}| \leq 1$

$S \rightarrow \bar{a} B A a$

$A \rightarrow b B | \epsilon.$

$\bar{a} B \rightarrow d$

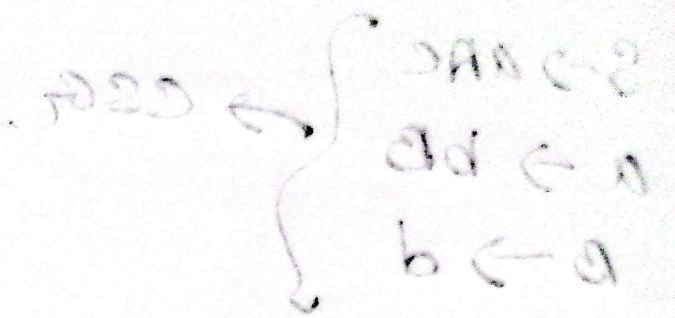
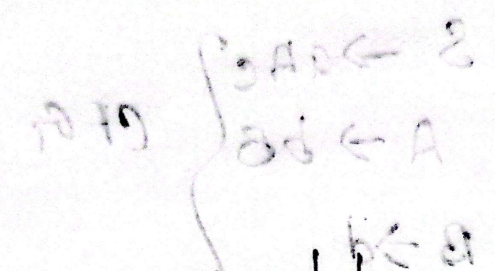
$\rightarrow CSG$ hobe na.

UG hobe.

$\hookrightarrow$ length is rule follow hobe nai.

① Inaccessible token.

length is rule



① Inaccessible token.

② Useless Variable / Non-terminal.

$S \rightarrow aA$

$A \rightarrow bB \mid \epsilon$

$C \rightarrow d$ Inaccessible token
 $B \rightarrow b$

Useless token.

$S \rightarrow a$
 $C \rightarrow a$
 $b \rightarrow a$

if else :

$S \rightarrow \text{if } (E) \text{ then } S.$

$\Rightarrow \text{if } (E) \text{ then } S \text{ else } S.$

$\Rightarrow S \text{ else if } (E) \text{ then } S$

$\rightarrow S \text{ else } S.$

*

if () { }

else if () { }.

else if () { }

else { }

E

$S \rightarrow \text{if } (E) \text{ then } S$

$\rightarrow \text{if } (E) \text{ then } S.$

1.1, 1.2, analysis.

Diagram of language processing system tools.

2.2 syntax definition. Example - 2.1, 2.2,

Fig. 2-3

ambiguous to unambiguous.

2.3 syntax directed definition.

Example - 2.6.

Fig-2.7. annotated parse tree:

2.4 parsing. , Recognizing identifier and keyword

3.1. Roles of lexical analyzer with block diagram.

↳ create table using lexeme, token and pattern.

3.3 .

Fig. 3.7.

Regular Expression (For C program)

Fig. 3.21, 3.23.